Here is a complete, self-contained Python script to fulfill the requirements of **Assignment 2.**

This solution includes sample data generation so that the notebook runs end-to-end and produces all required outputs. You can paste this entire script into your Jupyter Notebook (day2_cleaning.ipynb).

---

## Step 1: Create Sample Data, Load, and Explore the Dataset

Python

```
import pandas as pd
import numpy as np

# Creating a sample dataset with intentional quality issues
raw_data = {
    'Order_ID': [101, 102, 103, 104, 104, 105, 106, 107],
    'Order_Date': ['2023-01-15', '2023-01-16', 'INVALID_DATE',
'2023-01-18', '2023-01-18', '2023-01-19', '2023-01-20', '2023-01-21'],
    'Region': [' North ', 'South', 'East', ' West', ' West', 'North',
'South', 'East'],
    'Category': ['electronics', 'Furniture', 'ELECTRONICS',
'furniture', 'furniture', 'Office Supplies', 'furniture', 'Office
Supplies'],
    'Sales': [250.0, np.nan, 150.5, 400.0, 400.0, 75.0, 300.0, 120.0],
    'Quantity': [2, 1, -3, 4, 4, 5, 2, 1]
}

df_raw = pd.DataFrame(raw_data)
df_raw.to_csv('sales_data.csv', index=False)

# 1. Load and Explore
df = pd.read_csv('sales_data.csv')

print("=== HEAD ===")
print(df.head())

print("\n=== INFO ===")
print(df.info())

print("\n=== DESCRIBE ===")
print(df.describe(include='all'))

print("\n=== SHAPE ===")
print(df.shape)
```

---

## Steps 2 & 3: Identify Issues and Clean the Dataset

**Identified Data Quality Issues:**

1. **Missing Values**: NaNs in Sales.
2. **Duplicate Records**: Duplicate entries for Order_ID 104.
3. **Incorrect Data Types & Formats**: Order_Date contains string representation and invalid dates.
4. **Extra Spaces**: Leading/trailing spaces in Region.
5. **Inconsistent Text**: Mixed casing in Category (e.g., 'electronics' vs 'ELECTRONICS').
6. **Invalid Values**: Negative entries in Quantity.

Python

```
# Cleaning operations
df['Sales'] = df['Sales'].fillna(df['Sales'].median())
df = df.drop_duplicates()
df['Order_Date'] = pd.to_datetime(df['Order_Date'], errors='coerce')
df['Region'] = df['Region'].astype(str).str.strip()
df['Category'] = df['Category'].astype(str).str.strip().str.title()
df['Quantity'] = df['Quantity'].abs()

print("=== CLEANED DATA INFO ===")
print(df.info())
print("\n=== CLEANED DATASET ===")
print(df)
```

---

## Step 4: Perform Group By Analysis

Python

```
# Aggregate total sales by Region
region_sales = df.groupby('Region')['Sales'].sum().reset_index()

print("=== TOTAL SALES BY REGION ===")
print(region_sales)
```

---

## Step 5: Create a Pivot Table

Python

```
# Pivot table summarizing Sales by Region and Category
sales_pivot = df.pivot_table(
    index='Region',
    columns='Category',
    values='Sales',
    aggfunc='sum',
    fill_value=0
)
```

```
print("=== PIVOT TABLE: SALES BY REGION & CATEGORY ===")
print(sales_pivot)
```

---

## Step 6: Write a Data Quality Summary

```
### Data Quality Summary
- The original dataset contained missing values, duplicate entries,
incorrect date formats, and case inconsistencies.
- Data cleaning steps included median imputation, duplicate removal,
date parsing, text standardization, and sign correction.
- Missing values in the Sales column were successfully filled using
median imputation to preserve overall distribution.
- Duplicate records were dropped entirely, ensuring accurate count and
aggregation results across all categories.
- The cleaned dataset is structural, free of formatting anomalies,
correctly typed, and ready for further data analysis.
```

---

## Step 7: Save and Export

Python

```
# Export cleaned dataset
df.to_csv('cleaned_data.csv', index=False)
print("Dataset successfully saved to cleaned_data.csv")
```